



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

1

Primeiros Passos no Desenvolvimento Web: HTTP, REST e Flask

Resumo

Bem-vindo! Você está prestes a começar uma jornada para aprender como criar aplicativos web. Nesta primeira parte do curso, vamos aprender conceitos básicos da estrutura HTTP. Vamos explorar a arquitetura conhecida como *Rich Internet Applications* (RIAS). Em Rias, a função principal do servidor principal (e às vezes a única função) é fornecer ao aplicativo cliente a recuperação de dados e serviços de armazenamento. Neste modelo, o servidor se torna um serviço da web ou interface de programação de aplicativos (API). Existem vários protocolos pelos quais o RIAS pode se comunicar com um serviço da web. Protocolos de chamada de procedimento remoto (RPC), como XML-RPC ou seu derivado, o protocolo simplificado de acesso a objetos (SOAP) e ainda a arquitetura de transferência de estado representacional (REST) - favorito para aplicativos da web devido ao fato de ser construído sobre o modelo familiar do World Wide Web. Nesta unidade, também iremos abordar sobre a tecnologia de uma API RESTful.

“A simplicidade é o último grau de sofisticação.”

Leonardo da Vinci

Primeiros Passos na Programação Web com Python

Nesta jornada de **Programação Web Avançada**, começaremos pela base: entender como a **comunicação web** funciona e como podemos construir um **servidor HTTP** do zero utilizando apenas ferramentas nativas do **Python**.

Nosso foco será duplo:

- Compreender os **princípios fundamentais** da **comunicação HTTP**.
- Implementar um **servidor básico** com o **módulo** `http.server`, da **biblioteca padrão** do Python.

Esta abordagem não visa a produção imediata de aplicações completas, mas sim o entendimento dos **conceitos** por trás de **frameworks** modernos como **Flask** e **Django**.

Noções Básicas sobre Solicitações da Web

Quase toda **aplicação web** se baseia na **troca de informações** entre **clientes** (como navegadores) e **servidores**. Essa comunicação ocorre majoritariamente usando o protocolo HTTP — *Hypertext Transfer Protocol*.

Por exemplo, quando digitamos `uvv.br` no navegador, uma **requisição HTTP** é enviada ao **servidor**, que responde com uma **página HTML**. O **navegador** interpreta esse HTML e o exibe como uma página visual.

Essa troca é mediada por **métodos HTTP**, também conhecidos como **verbos HTTP**, como **GET**, **POST**, **PUT** e **DELETE**, que indicam a **intenção** da requisição. Por exemplo:

- **GET**: **Solicita** dados do servidor.
- **POST**: **Envia** dados ao servidor.

Todos os **servidores HTTP** devem, no mínimo, **implementar** o método **GET**. Já o **POST** é opcional, mas muito comum em aplicações mais complexas.

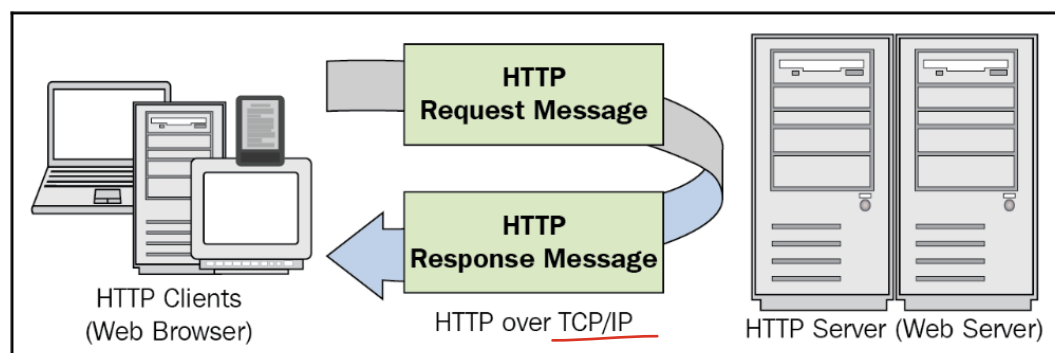


Figura 1.1: Diagrama de comunicação HTTP.

HTML — A Linguagem das Páginas Web

O conteúdo retornado pelo servidor geralmente é um arquivo **HTML** (HyperText Markup Language), que descreve a estrutura e o conteúdo da página. Veja um exemplo simples de HTML:

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Exemplo</title>
5 </head>
6 <body>
7   <h1>Bem-vindo a minha pagina</h1>
8   <p>Este eh um paragrafo de texto.</p>
9 </body>
10 </html>

```

Embora conhecer profundamente HTML não seja essencial para construir um servidor, é útil para entender o que está sendo transmitido entre cliente e servidor.

Códigos de Status HTTP

Cada resposta HTTP inclui um **código de status**, que indica o resultado da requisição. Alguns exemplos:

- **200**: Sucesso.
- **404**: Página não encontrada.
- **500**: Erro interno do servidor.

Esses códigos ajudam o cliente a interpretar corretamente o que aconteceu com a requisição enviada.

O Módulo `http.server` do Python

O Python oferece o módulo `http.server`, que permite a criação rápida de um servidor HTTP básico. Ele é ideal para fins **didáticos** e para **testes locais**, mas não deve ser usado em produção.

Estrutura do `http.server`

A criação de um servidor com `http.server` envolve duas classes principais:

- `HTTPServer`: cria o servidor, escuta uma porta e recebe conexões.
- `BaseHTTPRequestHandler`: trata as requisições recebidas.

Essa última é uma classe abstrata que exige que os métodos adequados sejam implementados, como:

- `do_GET(self)`
- `do_POST(self)`

Uma subclasse útil já implementada é a `SimpleHTTPRequestHandler`, que permite servir arquivos locais diretamente.

Ciclo de Vida de uma Requisição

O servidor escuta requisições usando o método `serve_forever`. Quando uma requisição é recebida:

1. O servidor aceita a conexão via socket.
2. Analisa os cabeçalhos da requisição.
3. Executa o método apropriado (por exemplo, `do_GET`).
4. Envia a resposta ao cliente.

Limitações do `http.server`

Apesar de útil para aprendizado, o `http.server` possui diversas limitações:

- Não suporta múltiplas conexões simultâneas (sem multithreading).
- Não oferece HTTPS, autenticação ou compressão de resposta.
- Não é compatível com a especificação **WSGI**¹.

Assim, aplicações feitas com `http.server` não funcionam facilmente com servidores modernos como **Gunicorn**² ou **uWSGI**.

Usos Comuns do `http.server`

Mesmo com essas restrições, o módulo é extremamente útil em contextos como:

- Servir arquivos estáticos em redes locais.
- Criar protótipos rápidos.

¹WSGI — Web Server Gateway Interface — é a interface padrão entre servidores web e frameworks Python, como Flask e Django. Mais em: <https://peps.python.org/pep-3333/>

²Gunicorn é um servidor HTTP WSGI para aplicações Python, amplamente usado em ambientes de produção.

- Simular APIs para testes.
- Ensinar como o protocolo HTTP funciona.

Exemplo de Implementação de um Servidor HTTP

Vamos agora implementar um servidor HTTP simples usando `http.server`. O exemplo abaixo cria um servidor que responde com uma mensagem simples ao receber uma requisição GET.

Código: `simple_server.py`

```
1 from http.server import HTTPServer, BaseHTTPRequestHandler
2
3 class MyHandler(BaseHTTPRequestHandler):
4     def do_GET(self):
5         self.send_response(200)
6         self.send_header("Content-type", "text/html")
7         self.end_headers()
8         message = "<html><body><h1>Ola, mundo!</h1></body></html>"
9         self.wfile.write(message.encode())
10
11 if __name__ == "__main__":
12     server_address = ('', 8000)
13     httpd = HTTPServer(server_address, MyHandler)
14     print("Servidor rodando em http://localhost:8000")
15     httpd.serve_forever()
```

Abra um terminal e execute o script. Depois, acesse `http://localhost:8000` em seu navegador. Você verá a mensagem “Olá, mundo!” renderizada na tela. Parabéns — você acabou de construir seu primeiro servidor web em Python!

Conclusão

Entender os princípios por trás de um servidor HTTP e como ele se comunica com navegadores e clientes é fundamental para qualquer programador web. Com esse conhecimento, você poderá dar passos mais avançados, como:

- Utilizar bibliotecas como Flask e FastAPI.
- Construir APIs RESTful.
- Compreender logs e falhas de comunicação em servidores.

Este é apenas o começo. Vamos continuar explorando as possibilidades da web com Python!

Exemplo de Servidor Básico

Para ilustrar o funcionamento básico de um servidor HTTP utilizando a biblioteca padrão do Python, apresentamos a seguir um exemplo minimalista de implementação de um servidor web. O código abaixo define uma classe chamada `MyWebServer`, cuja função principal é encapsular a inicialização e execução de um servidor HTTP local baseado na classe `HTTPServer`, pertencente ao módulo `http.server`.

A classe `MyWebServer` aceita, em sua construção, um manipulador de requisições HTTP (handler), o qual, por padrão, é o `SimpleHTTPRequestHandler`. Esse manipulador é responsável por interpretar e responder a requisições recebidas pelo servidor, tipicamente do tipo GET. O método `run` da classe é responsável por instanciar e iniciar o servidor, definindo o endereço e a porta onde ele ficará escutando requisições.

Esse exemplo inicial tem como objetivo demonstrar os componentes fundamentais necessários para criar um servidor HTTP funcional e fornece uma base para extensões futuras, como a personalização do comportamento do manipulador de requisições.

Código: `my_webserver.py`

```
1 import http.server as http_server
2
3 class MyWebServer:
4     def __init__(self, http_handler=http_server.SimpleHTTPRequestHandler) -> None:
5         self.http_handler = http_handler
6
7     def run(self, host="localhost", port=3001):
8         httpd = http_server.HTTPServer((host, port), self.http_handler)
9         print(f"Servidor Web rodando em {host}:{port}")
10        httpd.serve_forever()
```

A classe `SimpleHTTPRequestHandler` já implementa parte do comportamento esperado para verbos HTTP como GET, HEAD, etc. Para estender esse comportamento e manipular rotas específicas, podemos sobrescrever métodos como `do_GET`.

Código: main.py

```
1 from my_webserver import MyWebServer
2 from http.server import SimpleHTTPRequestHandler
3 import os
4
5 PORT = int(os.getenv("PORT", 3001)) # porta padrao 3001
6
7 class ManuseioHttpRequest(SimpleHTTPRequestHandler):
8     def do_GET(self):
9         if self.path == "/":
10             self.send_response(200)
11             self.send_header("Content-Type", "text/html; charset=utf-8")
12             self.end_headers()
13             self.wfile.write("<p>Ola Mundo!</p>".encode())
14
15         elif self.path == "/pagina1":
16             self.send_response(200)
17             self.send_header("Content-Type", "text/html; charset=utf-8")
18             self.end_headers()
19             self.wfile.write("<p>Essa seria uma rota para uma pagina!</p>".encode())
20
21         elif self.path == "/index":
22             self.send_response(200)
23             self.send_header("Content-Type", "text/html; charset=utf-8")
24             self.end_headers()
25             with open("index.html", "r") as f:
26                 res_body = f.read().format_map({"PORT": PORT})
27             self.wfile.write(res_body.encode())
28
29         else:
30             self.send_error(418) # "I'm a teapot" - codigo usado como exemplo
31
32 app = MyWebServer(ManuseioHttpRequest)
33
34 if __name__ == "__main__":
35     app.run()
```

Quando desenvolvemos aplicações web que respondem com HTML dinâmico, uma prática

comum é interpolar valores diretamente no corpo da resposta. Embora isso funcione em exemplos didáticos e controlados, **em ambientes reais essa prática representa um risco à segurança**, especialmente relacionado a **injeção de código malicioso**, como o **Cross-Site Scripting (XSS)**.

De acordo com a OWASP Foundation (2021) [1], a injeção de scripts maliciosos no lado do cliente (como XSS) permanece entre os dez riscos mais críticos de segurança em aplicações web, sendo comumente explorada através de falhas de validação e interpolação insegura de dados.

Esse exemplo marca um ponto de partida essencial para a construção de servidores web personalizados, além de introduzir preocupações fundamentais de segurança, que se tornam críticas à medida que a aplicação evolui.

Exemplo de HTML Renderizado

Um modelo de arquivo `.html` também será utilizado nesse exemplo introdutório. Note que neste caso, a classe `SimpleHTTPRequestHandler` irá renderizar o arquivo `index.html`, tal que o nosso servidor simplesmente retorna o conteúdo daquele arquivo. E, caso façamos alguma alteração no interior do arquivo `index.html`, a página será renderizada após ser recarregada no servidor.

Código: `index.html`

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Aula Web Server</title>
5 </head>
6
7 <body>
8     <h1>Nosso servidor HTTP esta rodando na Porta: {PORT}</h1>
9 </body>
10 </html>
```

Boas Práticas: WSGI e a PEP 3333

Embora a abordagem anterior seja didática, aplicações reais em Python utilizam uma interface padronizada entre servidores e frameworks chamada **WSGI (Web Server Gateway Interface)**, definida na PEP 3333.

Essa especificação estabelece uma interface entre servidores web (como Gunicorn, uWSGI ou o próprio `wsgiref`) e aplicações Python, permitindo que qualquer framework compatível funcione de forma intercambiável com diferentes servidores.

Implementar uma aplicação compatível com WSGI é o próximo passo lógico caso o objetivo seja construir um framework reutilizável e robusto.

Filosofia do Python

A criação de frameworks e bibliotecas em Python segue princípios estabelecidos na PEP 20 – The Zen of Python, tais como:

- *Explícito é melhor do que implícito;*
- *A simplicidade é melhor do que a complexidade;*
- *Legibilidade conta.*

Além disso, é recomendável seguir o guia oficial de estilo de código Python, descrito na PEP 8.

Algumas Considerações

Até aqui, foi explorado o módulo `http.server` da biblioteca padrão do Python como uma ferramenta educativa para a compreensão do protocolo HTTP e da construção de servidores web básicos. Embora adequado para fins didáticos e prototipagem, este módulo não

se destina a ambientes de produção devido às suas limitações em termos de desempenho, segurança e funcionalidades avançadas.

Para garantir boas práticas no desenvolvimento de aplicações web robustas, a comunidade Python estabelece e documenta recomendações e padrões técnicos através das *Python Enhancement Proposals* (PEPs - Propostas de aprimoramento do Python). As PEPs são documentos oficiais que descrevem novas funcionalidades, processos, diretrizes ou padrões para a linguagem e seu ecossistema, incluindo protocolos e interfaces fundamentais, como é o caso da PEP 3333, que define a especificação WSGI (Web Server Gateway Interface).

A WSGI, formalizada pela PEP 3333, representa um padrão essencial para a interoperabilidade entre servidores HTTP e aplicações Python, possibilitando a criação de frameworks modernos e servidores web eficientes. A existência das PEPs, portanto, é fundamental para orientar o desenvolvimento e a evolução do Python, garantindo coerência, compatibilidade e melhores práticas para toda a comunidade, especialmente em áreas tão críticas quanto o desenvolvimento web.

Por fim, para projetos que demandam escalabilidade, segurança e suporte a múltiplos protocolos, recomenda-se a adoção de frameworks e servidores especializados, como Flask, FastAPI ou Django, que seguem as diretrizes estabelecidas pelas PEPs e proporcionam um ambiente estável e otimizado para aplicações em produção.

Introdução ao Estilo Arquitetônico REST

À medida que aprofundamos a construção de servidores web e a manipulação direta de requisições HTTP utilizando a linguagem Python, torna-se fundamental compreender as diretrizes arquitetônicas que sustentam os serviços web modernos. Nesse contexto, destaca-se o estilo arquitetônico conhecido como **REST** (*Representational State Transfer*), amplamente adotado no desenvolvimento de APIs que operam sobre o protocolo HTTP.

REST não é um protocolo, mas sim um conjunto de princípios que definem uma abordagem padronizada para a comunicação entre sistemas distribuídos. Esse estilo arquitetônico foi formalizado por Roy Fielding em sua tese de doutorado³, e orienta o *design* de aplicações que aproveitam a estrutura e os recursos da Web de maneira eficiente, escalável e interoperável.

Uma aplicação web baseada em REST deve seguir seis restrições fundamentais. A primeira delas é a **separação entre cliente e servidor** (*client-server*), o que implica que a interface de comunicação entre os dois deve ser bem definida e desacoplada da lógica de negócio ou da persistência de dados. A segunda é a **ausência de estado** (*statelessness*): cada requisição do cliente ao servidor deve conter todas as informações necessárias para seu processamento, sem depender de contexto armazenado entre requisições consecutivas.

A terceira restrição é o **suporte a cache**, que permite que respostas do servidor sejam rotuladas de forma que clientes ou intermediários (como proxies) possam armazená-las temporariamente, reduzindo latência e carga sobre o servidor. A quarta característica – considerada central ao estilo REST – é a **existência de uma interface uniforme**, que define regras consistentes de interação, incluindo a utilização de identificadores únicos para recursos, mensagens autodescritivas, representações padronizadas (como JSON ou XML) e navegação por hipermídia.

Complementando essas diretrizes, REST permite que as **arquiteturas sejam organizadas em camadas** (*layered system*), de modo que componentes intermediários, como proxies, balanceadores de carga ou gateways, possam ser inseridos entre cliente e servidor para

³Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine. Disponível em: <https://roy.gbiv.com/pubs/dissertation/top.htm>.

fins de escalabilidade, segurança e confiabilidade. Finalmente, a última restrição, opcional, é o suporte a **código sob demanda**, onde o servidor pode fornecer ao cliente scripts ou funcionalidades adicionais a serem executadas dinamicamente.

No modelo REST, o conceito de *recurso* é central. Um recurso representa qualquer entidade de interesse no domínio da aplicação, como usuários, produtos, postagens ou comentários. Cada recurso é identificado unicamente por meio de uma URL (*Uniform Resource Locator*), que funciona como um endereço acessível via protocolo HTTP. Por exemplo, em um sistema de blog, uma postagem específica poderia ser acessada via `/api/posts/12345`, onde 12345 representa o identificador único da postagem, equivalente a uma chave primária em um banco de dados relacional.

Além dos recursos individuais, REST define o uso de coleções de recursos. O endpoint `/api/posts/`, por exemplo, pode representar todas as postagens existentes, enquanto `/api/comments/` pode referenciar o conjunto total de comentários. É igualmente possível definir URLs que correspondam a subconjuntos lógicos, como `/api/posts/12345/comments/`, que retornaria todos os comentários associados à postagem identificada por 12345.

Por convenção, URLs que representam coleções de recursos terminam com uma barra ("`/`"), indicando sua semântica como diretórios ou agrupamentos lógicos. Essa prática favorece a clareza na modelagem de recursos e contribui para a previsibilidade das interfaces **RESTful**.

O adjetivo **RESTful** é utilizado para descrever sistemas ou APIs que seguem de forma coerente e consistente os princípios do estilo arquitetônico REST. Em outras palavras, uma aplicação é considerada *RESTful* quando respeita as restrições impostas por REST, como a separação entre cliente e servidor, ausência de estado entre as requisições (*statelessness*), suporte a cache, interface uniforme, sistema em camadas e, opcionalmente, código sob demanda. O termo não é um padrão formal ou absoluto, mas um indicativo de aderência conceitual à filosofia REST. Assim, ao descrever uma API como sendo RESTful, estamos afirmando que ela foi projetada para funcionar de acordo com essas diretrizes, o que favorece escalabilidade, simplicidade, interoperabilidade e a reutilização dos recursos e estruturas da Web de forma natural.

A compreensão desses princípios é essencial não apenas para o uso de frameworks web

em Python – como Flask ou Django REST Framework – mas também para a correta estruturação de servidores HTTP que almejam seguir boas práticas de interoperabilidade e escalabilidade. Ao implementar um servidor básico com `http.server`, como fizemos anteriormente, abre-se a oportunidade de observar, em nível baixo, como essas abstrações são construídas a partir do protocolo HTTP e como podem ser estendidas em direção a uma API RESTful completa.

O exemplo a seguir representa uma etapa intermediária entre a implementação de um servidor HTTP básico e a construção de uma API RESTful completa. Utilizando apenas a biblioteca padrão do Python, definimos uma classe derivada de `BaseHTTPRequestHandler` para manipular os métodos GET, POST e DELETE, simulando operações típicas sobre recursos REST, neste caso, uma lista de usuários mantida em memória. Este tipo de implementação permite observar, com clareza, como os verbos HTTP são mapeados para ações na aplicação, como são tratados os códigos de status HTTP e como as rotas seguem uma estrutura hierárquica baseada em identificadores de recursos. Embora rudimentar e sem suporte a persistência ou autenticação, esse exemplo fornece uma base concreta para compreender as abstrações que frameworks como Flask ou Django encapsulam.

Código: `rest_server.py`

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2 import json
3
4 # Simulacao de um "banco de dados" em memoria
5 USERS = [
6     {"id": 1, "name": "Alice"},
7     {"id": 2, "name": "Bob"}
8 ]
9
10 class RestHTTPRequestHandler(BaseHTTPRequestHandler):
11     def _set_headers(self, code=200, content_type="application/json"):
12         self.send_response(code)
13         self.send_header("Content-Type", content_type)
14         self.end_headers()
15
16     def do_GET(self):
17         if self.path == "/api/users":
```

```

18         self._set_headers()
19         self.wfile.write(json.dumps(USERS).encode())
20     elif self.path.startswith("/api/users/"):
21         try:
22             user_id = int(self.path.split("/")[-1])
23             user = next((u for u in USERS if u["id"] == user_id), None)
24             if user:
25                 self._set_headers()
26                 self.wfile.write(json.dumps(user).encode())
27             else:
28                 self._set_headers(404)
29                 self.wfile.write(json.dumps({"error": "User not found"}).encode
30                                     ())
31         except ValueError:
32             self._set_headers(400)
33             self.wfile.write(json.dumps({"error": "Invalid user ID"}).encode())
34     else:
35         self._set_headers(404)
36         self.wfile.write(json.dumps({"error": "Route not found"}).encode())
37
38     def do_POST(self):
39         if self.path == "/api/users":
40             content_length = int(self.headers["Content-Length"])
41             post_data = self.rfile.read(content_length)
42             try:
43                 new_user = json.loads(post_data)
44                 new_user["id"] = max(u["id"] for u in USERS) + 1 if USERS else 1
45                 USERS.append(new_user)
46                 self._set_headers(201)
47                 self.wfile.write(json.dumps(new_user).encode())
48             except json.JSONDecodeError:
49                 self._set_headers(400)
50                 self.wfile.write(json.dumps({"error": "Invalid JSON"}).encode())
51
52     def do_DELETE(self):
53         if self.path.startswith("/api/users/"):
54             try:
55                 user_id = int(self.path.split("/")[-1])
56                 global USERS

```

```
56         USERS = [u for u in USERS if u["id"] != user_id]
57         self._set_headers(204)
58     except ValueError:
59         self._set_headers(400)
60         self.wfile.write(json.dumps({"error": "Invalid user ID"}).encode())
61     else:
62         self._set_headers(404)
63         self.wfile.write(json.dumps({"error": "Route not found"}).encode())
64
65     def run(server_class=HTTPServer, handler_class=RestHTTPRequestHandler, port
66             =3001):
67         server_address = ("", port)
68         httpd = server_class(server_address, handler_class)
69         print(f"Servidor REST rodando na porta {port}")
70         httpd.serve_forever()
71
72 if __name__ == "__main__":
73     run()
```

Como antes, o primeiro passo é salvar o código acima em um arquivo chamado `rest_server.py`. Certifique-se de ter o Python 3 instalado e execute o seguinte comando no terminal:

```
python rest_server.py
```

Após a execução, o terminal deverá exibir a seguinte mensagem:

```
Servidor REST rodando na porta 3001
```

Isso indica que o servidor está pronto para receber requisições HTTP na porta 3001 do `localhost`.

Agora, uma forma prática de interagir com a API é utilizar a ferramenta de linha de comando `curl`, disponível em sistemas Linux, macOS e, recentemente, também no Windows. Abaixo listamos exemplos de comandos que podem ser utilizados para interagir com as rotas do servidor.

Listar todos os usuários cadastrados

```
curl http://localhost:3001/api/users
```

Resposta esperada (200 OK):

```
[  
  {"id": 1, "name": "Alice"},  
  {"id": 2, "name": "Bob"}  
]
```

Buscar um usuário específico

```
curl http://localhost:3001/api/users/1
```

Resposta esperada (200 OK):

```
{"id": 1, "name": "Alice"}
```

Se o ID não existir, será retornado um erro:

```
{"error": "User not found"}
```

Criar um novo usuário

```
curl -X POST -H "Content-Type: application/json" \  
  -d '{"name": "Charlie"}' \  
  http://localhost:3001/api/users
```

Resposta esperada (201 Created):

```
{"name": "Charlie", "id": 3}
```

Após essa operação, o usuário Charlie passa a fazer parte da lista de usuários.

Deletar um usuário existente

```
curl -X DELETE http://localhost:3001/api/users/2
```

Resposta esperada (204 No Content):

O servidor indica que a exclusão foi realizada com sucesso, mas não retorna conteúdo.

Requisições com erro

Testar casos de erro também é importante:

```
curl http://localhost:3001/api/users/abc
```

Resposta esperada (400 Bad Request):

```
{"error": "Invalid user ID"}
```

Outro exemplo é tentar acessar uma rota inexistente:

```
curl http://localhost:3001/usuarios
```

Resposta esperada (404 Not Found):

```
{"error": "Route not found"}
```

Conclusão da experimentação

Essa prática oferece uma visão clara de como uma API RESTful pode ser construída a partir do zero, sem o uso de frameworks externos. Ao analisar as respostas do servidor, espero que tenha compreendido como os métodos HTTP são utilizados para manipular recursos, como os códigos de status informam o resultado das operações e como as rotas podem ser organizadas em conformidade com o estilo REST.

Além disso, observar a estrutura de mensagens JSON nas respostas reforça a importância de padrões de interoperabilidade no desenvolvimento de sistemas distribuídos. Tais experimentos são fundamentais para compreender os conceitos que futuramente serão abstraídos em frameworks como Flask ou Django REST Framework.

Organização de Aplicações Reais: Introdução aos *Blueprints*

Agora que compreendemos como um servidor HTTP pode ser construído do zero utilizando bibliotecas nativas e como o padrão WSGI promove a interoperabilidade em aplicações Python, é o momento de dar uma primeira olhada em como frameworks modernos como o Flask estruturam uma aplicação web de forma modular e escalável.

Um dos recursos centrais nesse processo de organização é o uso de um componente chamado **Blueprint**.

O que é um Blueprint?

No Flask, um **Blueprint** é uma forma de organizar o código da aplicação em partes modulares e reutilizáveis. Ele funciona como uma planta (“blueprint”) de uma funcionalidade que pode ser registrada na aplicação principal. Por exemplo:

- Autenticação (auth)
- Administração (admin)
- API (api)

Por que usar Blueprints?

O uso de Blueprints no Flask oferece uma abordagem modular para a construção de aplicações web, promovendo uma série de boas práticas de engenharia de software. A principal vantagem é a separação de responsabilidades, permitindo que diferentes partes da aplicação (como autenticação, administração, API, entre outras) sejam implementadas de forma isolada, facilitando o desenvolvimento em equipe e o controle de versões.

Além disso, Blueprints contribuem diretamente para uma melhor organização do código-fonte, reduzindo a complexidade da aplicação principal e tornando o projeto mais legível e compreensível. Essa modularização também favorece a reutilização de componentes, já

que cada Blueprint pode ser transferido entre projetos com mínima adaptação, economizando tempo de desenvolvimento e promovendo consistência.

Outro benefício importante é a escalabilidade, pois a estrutura modular facilita a adição de novas funcionalidades sem comprometer a integridade das partes existentes. Do ponto de vista da manutenção, alterações localizadas em um Blueprint tendem a ser menos propensas a causar efeitos colaterais indesejados no restante da aplicação, promovendo maior robustez ao sistema como um todo.

De acordo com Grinberg (2018), o uso de Blueprints é altamente recomendado em aplicações Flask de médio e grande porte, pois "permite dividir uma aplicação em componentes menores que podem ser desenvolvidos e testados de forma independente" [2]. A seguir, temos uma visão geral da nossa proposta de introdução ao conceito que iremos implementar.

```
meu_app/  
    app.py  
    calculadora/  
        __init__.py  
        rotas.py
```

Código: calculadora/rotas.py

```
1 from flask import Blueprint, jsonify  
2 calculadora_bp = Blueprint('calculadora', __name__)  
3  
4 @calculadora_bp.route('/soma/<int:a>/<int:b>')  
5 def soma(a, b):  
6     resultado = a + b  
7     return jsonify(  
8         'operacao': 'soma',  
9         'a': a,  
10        'b': b,  
11        'resultado': resultado  
12    })
```

Código: calculadora/__init__.py

```
1 from .rotas import calculadora_bp
```

Código: app.py

```
1 from flask import Flask
2 from calculadora import calculadora_bp
3
4 app = Flask(__name__)
5 app.register_blueprint(calculadora_bp, url_prefix='/calc')
6
7 if __name__ == '__main__':
8     app.run(debug=True)
```

- Executar:

```
python app.py
```

- Acessar no navegador ou via curl:

```
http://localhost:5000/calc/soma/7/3
```

Saída JSON esperada

```
{
    "operacao": "soma",
    "a": 7,
    "b": 3,
    "resultado": 10
}
```

Explicação

- O Blueprint calculadora define a rota `/soma`.
- A aplicação principal registra o blueprint com o prefixo `/calc`.
- A URL completa se torna `/calc/soma/{a}/{b}`.

Este exemplo serve como introdução à organização de aplicações reais usando o Flask, e será expandido nas próximas seções, onde abordaremos o framework de maneira mais detalhada.

Exercício Proposto

- Criar nova rota: `/subtrai/<int:a>/<int:b>`
- Retornar resultado da subtração no mesmo formato JSON
- (Desafio) Criar um novo blueprint para operações de multiplicação

Lista de Exercícios

1.1 Qual das opções representa corretamente o principal objetivo do módulo `http.server` no Python?

- (A) Servir aplicações Flask em ambiente de produção
- (B) Criar APIs RESTful completas com suporte a autenticação
- (C) Construir interfaces gráficas para aplicações desktop
- (D) Permitir a criação de servidores HTTP básicos para testes e fins didáticos
- (E) Compilar código Python em bytecode nativo

1.2 Qual das opções abaixo corresponde ao método responsável por manter o servidor HTTP em execução contínua no `http.server`?

- (A) `run_forever()`
- (B) `accept_connections()`
- (C) `serve_forever()`
- (D) `start_server()`
- (E) `handle_requests()`

1.3 O código abaixo define uma rota `/soma` utilizando Flask. Qual será a saída JSON ao acessar `/calc/soma/10/5`?

```
1 @calculadora_bp.route('/soma/<int:a>/<int:b>')
2 def soma(a, b):
3     resultado = a + b
4     return jsonify({'a': a, 'b': b, 'resultado': resultado})
```

- (A) `{ "a": 10, "b": 5, "resultado": 5 }`
- (B) `{ "a": 10, "b": 5, "resultado": 15 }`
- (C) `{ "a": 10, "b": 5, "resultado": 50 }`
- (D) `{ "a": 10, "b": 5, "resultado": "10 + 5" }`

(E) { "resultado": null }

1.4 Sobre o padrão WSGI, é correto afirmar que:

- (A) É um módulo do Flask que permite fazer deploy em servidores Gunicorn
- (B) É um protocolo de rede baseado em REST
- (C) Substitui completamente o uso do `http.server`
- (D) Permite criar APIs em JavaScript com suporte a sockets
- (E) É uma interface entre servidores web e aplicações Python

1.5 Qual código abaixo registra corretamente um blueprint no Flask?

- (A) `app.include_module('calc')`
- (B) `app.use_blueprint(calc, prefix="/calc")`
- (C) `app.mount(calc_bp)`
- (D) `app.register_blueprint(calculadora_bp, url_prefix='/calc')`
- (E) `app.route('/calc', blueprint=calculadora_bp)`

1.6 A classe `BaseHTTPRequestHandler` do módulo `http.server` é usada para:

- (A) Manusear diretamente requisições HTTP como GET e POST
- (B) Configurar servidores de banco de dados
- (C) Criar páginas HTML com formulários
- (D) Construir servidores seguros com suporte a HTTPS
- (E) Fazer parsing de arquivos XML

1.7 Quais são as principais vantagens de utilizar Blueprints em aplicações Flask?

- (A) Otimizar o uso de memória e CPU
- (B) Reutilização de código, separação de responsabilidades e escalabilidade
- (C) Garantir compatibilidade com servidores Apache e Nginx
- (D) Reduzir o número de dependências externas no projeto
- (E) Proteger rotas da aplicação com autenticação OAuth

1.8 Assinale a alternativa que **não** representa uma limitação do módulo `http.server`:

- (A) Ausência de suporte nativo a HTTPS
- (B) Incompatibilidade com a especificação WSGI
- (C) Falta de suporte a manipulação de cookies
- (D) Suporte total à escalabilidade em produção
- (E) Ausência de suporte a compressão de respostas

1.9 No código abaixo, qual comportamento é esperado ao acessar uma rota diferente das definidas?

```

1 def do_GET(self):
2     if self.path == "/":
3         self.send_response(200)
4         ...
5     else:
6         self.send_error(418)

```

- (A) Retornar erro HTTP 404
- (B) Exibir uma página HTML padrão do Flask
- (C) Retornar o código "I'm a teapot"(418)
- (D) Fazer redirecionamento automático para "/"
- (E) Lançar uma exceção no terminal

1.10 Qual das opções corresponde à principal motivação da criação do padrão WSGI (PEP 3333)?

- (A) Fornecer suporte nativo a autenticação em APIs
- (B) Permitir integração entre qualquer servidor HTTP e frameworks Python
- (C) Aumentar a velocidade de servidores baseados em Flask
- (D) Criar uma nova linguagem de definição de rotas
- (E) Otimizar servidores com suporte multithread

Referências Bibliográficas

- [1] FOUNDATION, O. *OWASP Top 10 – 2021: The Ten Most Critical Web Application Security Risks*. Open Worldwide Application Security Project, 2021. Acesso em: 19 ago. 2025. Disponível em: <<https://owasp.org/Top10/>>.
- [2] GRINBERG, M. *Flask Web Development: Developing Web Applications with Python*. 2. ed. Sebastopol: O'Reilly Media, 2018. ISBN 9781491991732.